Git RefLog and Time travelling

→ lets see a Senario where we accidentaly deleted a file and Commited

→ But we need that file That is where Time Travelling comes to picture

difference between git log and reflog

→ both are similar Commands but reflog will have a detailed description.

» git log --oneline

```
PS D:\Git_Mastery\Git_Practiece> git log --oneline
79ed95c (HEAD -> main, test_rebase_1) Adding data from
rebase_1 to main
b1744fd Adding Data to main.txt form main
c6b24a8 Adding Data to Main
cc701f1 (branch_1) From Branch_1
70e14a6 From main
1cd28be Conflicts Are Solved
e5cfe81 Fixing main.txt from main
d26a212 (bug_fix) Fixing main.txt from bug_fix
7c2d215 Adding data to main.txt
4c413c0 Merging main and Feature_2
d579e77 merging Main and Feature_1
16e413f (feature_1) Adding new Data to feature_1
8cd8f66 Adding new Data to feature_1
d9ea32b Adding more Data to Main
dd85bbc (feature_2) Adding data to feature_2
c21d859 Adding More Data to main Page
15c4228 Adding data for Feature_1
7ef0910 Adding data to File1
816e3ed Initial Commit
PS D:\Git_Mastery\Git_Practiece>
```

» git reflog --oneline

```
PS D:\Git_Mastery\Git_Practiece> git reflog --oneline
79ed95c (HEAD -> main, test_rebase_1) HEAD@{0}: merge test_rebase_1: Fast-forward
79ed95c (HEAD -> main, test_rebase_1) HEAD@{0}: merge test_rebase_1: Fast-forward
b1744fd HEAD@{1}: checkout: moving from test_rebase_1 to main
79ed95c (HEAD -> main, test_rebase_1) HEAD@{2}: rebase (finish): returning to refs/heads/test_rebase_1
79ed95c (HEAD -> main, test_rebase_1) HEAD@{3}: rebase (continue): Adding data from rebase_1 to main
b1744fd HEAD@{4}: rebase (start): checkout main
7e68707 HEAD@{5}: checkout: moving from main to test_rebase_1
b1744fd HEAD@{6}: checkout: moving from main to main
b1744fd HEAD@{7}: commit: Adding Data to main.txt form main
c6b24a8 HEAD@{8}: checkout: moving from main to main
c6b24a8 HEAD@{9}: checkout: moving from test_rebase_1 to main
7e68707 HEAD@{10}: commit: Adding data from rebase_1 to main
c6b24a8 HEAD@{11}: checkout: moving from main to test_rebase_1
c6b24a8 HEAD@{12}: commit: Adding Data to Main
cc701f1 (branch_1) HEAD@{13}: merge branch_1: Fast-forward
70e14a6 HEAD@{14}: checkout: moving from branch_1 to main
cc701f1 (branch_1) HEAD@{15}: rebase (finish): returning to refs/heads/branch_1
cc701f1 (branch_1) HEAD@{16}: rebase (pick): From Branch_1
70e14a6 HEAD@{17}: rebase (start): checkout main
5a35318 HEAD@{18}: checkout: moving from main to branch_1
70e14a6 HEAD@{19}: commit: From main
1cd28be HEAD@{20}: checkout: moving from branch_1 to main
5a35318 HEAD@{21}: commit: From Branch_1
1cd28be HEAD@{22}: checkout: moving from main to branch_1
1cd28be HEAD@{23}: commit (merge): Conflicts Are Solved
e5cfe81 HEAD@{24}: checkout: moving from main to main
e5cfe81 HEAD@{25}: commit: Fixing main.txt from main
7c2d215 HEAD@{26}: checkout: moving from bug_fix to main
d26a212 (bug_fix) HEAD@{27}: commit: Fixing main.txt from bug_fix
7c2d215 HEAD@{28}: checkout: moving from main to bug_fix
7c2d215 HEAD@{29}: commit: Adding data to main.txt
4c413c0 HEAD@{30}: merge feature_2: Merge made by the 'ort' strategy.
d579e77 HEAD@{31}: merge feature_1: Merge made by the 'ort' strategy.
d9ea32b HEAD@{32}: checkout: moving from feature_1 to main
16e413f (feature_1) HEAD@{33}: commit: Adding new Data to feature_1
8cd8f66 HEAD@{34}: commit: Adding new Data to feature_1
15c4228 HEAD@{35}: checkout: moving from main to feature_1
d9ea32b HEAD@{36}: commit: Adding more Data to Main
c21d859 HEAD@{37}: checkout: moving from feature_2 to main
dd85bbc (feature_2) HEAD@{38}: commit: Adding data to feature_2
c21d859 HEAD@{39}: checkout: moving from main to feature_2
c21d859 HEAD@{40}: commit: Adding More Data to main Page
7ef0910 HEAD@{41}: checkout: moving from feature_1 to main
15c4228 HEAD@{42}: commit: Adding data for Feature_1
```

→ There is some extra information will be in reflog

→ all comit / branch comit / merge / rebase everything

→ Now if we want to jump back to any comit we can go by

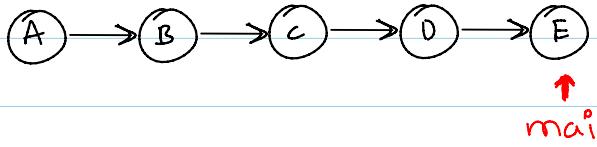
commitid → 79ed95c

Head → HEAD@{11}

→ there are 3 ways to do that

1) git reset --hard <commit_id> / <Head@{}>

→ Suppose we have graph like this



→ if we want to jump back to (C) Then we should use The Command

>> git reset --hard c



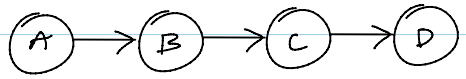
→ Now The (D) and (E) are in the git object database after few days they get deleted

→ what ever Commits we do again start from (C) as a fresh branch

(2) git revert

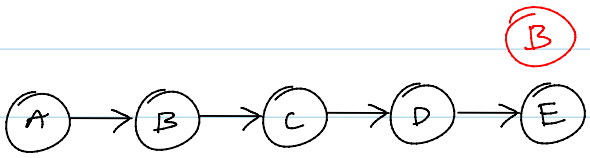
→ Same like reset but when we pushed the code to github we will use Revert

→ for example



→ now i wanted to get back to (B)

>> git revert B

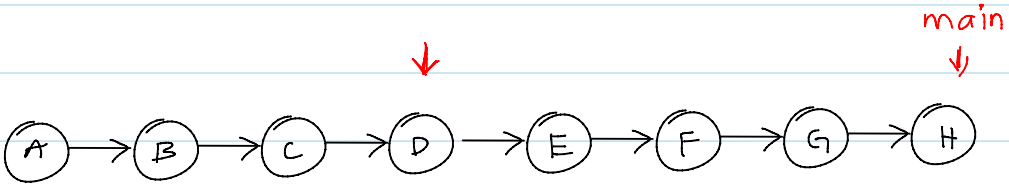


But that atchully having (B) commit data

(3) git checkout

→ by using this we can move to that particular commit but main directly pointing to lastonly

>> git checkout <commit Id>



→ heare head will be detached and points to one place so if we want to make any changes we should create a new branch from here

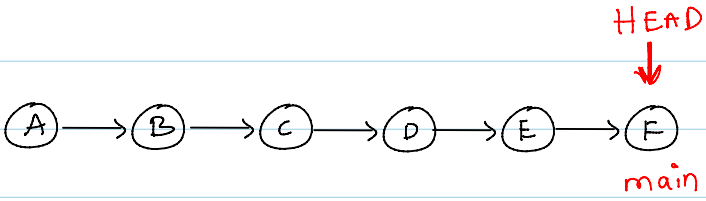
→ and we can merge that later

Important Commands

1) >> git diff → difference between working directory and Staging Area
→ what new files added / removed

2) >> git reset → when we added files to staging area this Command will remove those.

Imp Imp about **time travel**



1) when you do >> git reset B -- Something

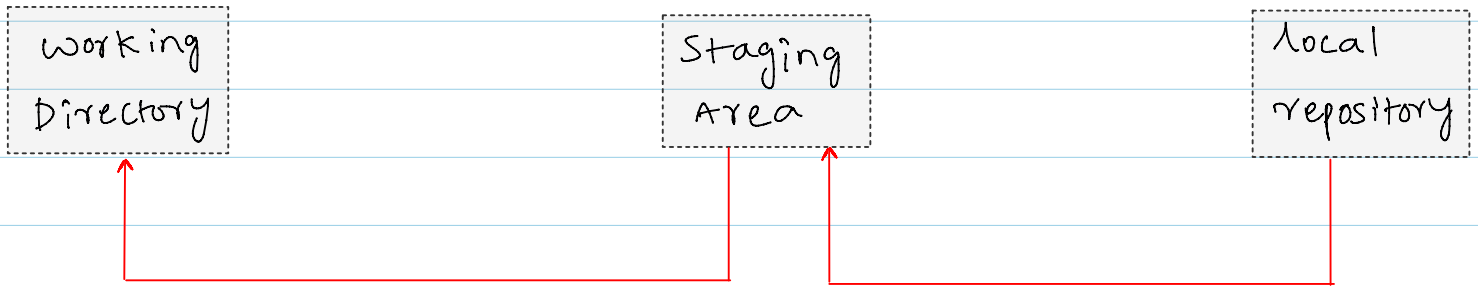
→ few things can be possible.

-- Soft

→ all Commits after B are come to Staging Area

→ if you Commit again from B (C)(D)(E)(F) will be gone and a new Commit will be created after (B)

→ when we run -- soft reset all the commits after the commits are restored and come to staging Area



→ git restore --staged .

git restore <file>

→ to restore Deleted file/s

getting from staging Area to working Directory

git reset --soft <c_id>

→ it will come to the Staging Area of that commit

→ all Commits That we are doing will be recorded as a fresh Commit from now in Soft Reset ~~Soft Reset~~

--hard → if we do this head will move to (B) and all commits after (B) are deleted

→ when we do a **hard Reset** to a specific commit

→ The commits after that will be deleted and branch (HEAD) will come to the selected branch

⇒ all file changes (update / Delete / any) change will be reset to That specific **Commit id**

--mixed → if we do this Head pointer will move to (B) and all changes there in map

→ if we do a **mixed Reset** to a Specific Commit id (HEAD and Branch) will come back to That Commit id

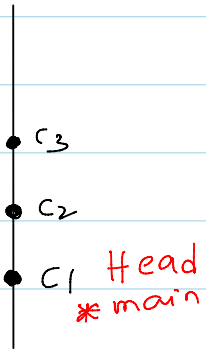
→ But the file changes (add / Delete / modify) in The commits after The specific Commit will be There They won't be changed.

git reset

→ This was done when our branch is under local repository and move back to any commit

→ if we do reset the HEAD and branch will move to the specific commit_id

>> git reset c1

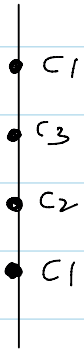


revert

→ This was done when our branch is already pushed to github and we want to goback

→ if we revert changes to a specific commit-id a new comit with copy data of that Commit_id will come to top
(it will create new commit

>> git revert c1



→ when working with Time travel **reflog** is imp

⇒ when we do time travel if we want a deleted file back Then do

>> git restore <file.txt>

>> git restore --staged . (or) <File name>

↳ restore from staging Area.